

Metaheurísticas MUIA

Non-dominated sorting genetic algorithm II

Noising methods

Hormigas Max-Min

Janire Amesti Soler

Aritz Bermejo Canal

Juan García Santos

Índice

1	Resumen.....	3
2	Non-dominated sorting genetic algorithm II	4
2.1	Introducción	4
2.2	NSGA-II algorithm.....	5
2.2.1	Fast non-dominated Sorting Approach (Intensificación)	5
2.2.2	Diversity preservation (Diversificación)	5
2.2.3	Main loop	6
2.3	Implementación	7
2.4	Conclusiones	9
3	Noising methods	10
3.1	Introducción	10
3.2	Procedimiento.....	10
3.2.1	Parámetros de los métodos de ruido.....	10
3.2.2	Estrategias de introducción de ruido	10
3.2.3	Estrategias de decrecimiento del ruido	12
3.2.4	Exploración de los entornos.....	12
3.3	Variantes.....	13
3.4	Aplicaciones	14
3.5	Conclusiones	14
4	Hormigas Max-Min.....	15
4.1	Introducción	15
4.2	Algoritmo Hormigas Max-Min.....	15
4.2.1	Pheromone trail updating	15
4.2.2	Pheromone trail limits.....	16
4.2.3	Inicialización de rastro de feromonas	17
4.3	Implementación	18
4.4	Conclusiones	21
5	Conclusiones	22
6	Bibliografía	23

1 Resumen

En la presente memoria se plantean los tres trabajos realizados. Se analizan teóricamente tres metaheurísticas, NSGA-II, Hormigas MaxMin y Noising Methods. En las dos primeras, que se tratan de un algoritmo evolutivo y una metaheurística constructiva, se realiza, además, una implementación sobre dos problemas diferentes. Las tres secciones, correspondientes a cada algoritmo, cuentan con la misma estructura: una introducción de la metaheurística, la explicación teórica, la implementación con sus resultados en caso de haberla y finalmente las conclusiones propias de cada sección. Finalmente, el trabajo finaliza con unas conclusiones generales del trabajo realizado.

2 Non-dominated sorting genetic algorithm II

2.1 Introducción

Los algoritmos evolutivos (AE) constituyen una familia de técnicas de optimización y búsqueda inspirada en los principios de la evolución biológica. Estos algoritmos se han vuelto fundamentales en la resolución de problemas complejos en diversos campos. Su popularidad se debe a su capacidad para encontrar soluciones eficientes y adaptativas a través de procesos de selección natural y operadores genéticos.

Non-Dominated Sorting Genetic Algorithm II (NSGA-II) [1] es un algoritmo genético dentro de la familia de algoritmos evolutivos, diseñado para abordar problemas específicos, especialmente en el ámbito de la optimización multiobjetivo.

Los algoritmos evolutivos siguen la siguiente estructura general:

Paso 1: Generar la población inicial de individuos. (Primera generación)

Paso 2: Evaluar la aptitud de la primera generación.

Paso 3: Repetir los siguientes pasos generacionales hasta la terminación.

3.1. Seleccionar individuos para la reproducción. (Padres)

3.2. Procrear nuevos individuos mediante operaciones de cruce y mutación para obtener la descendencia.

3.3. Evaluar la aptitud individual de los nuevos individuos.

3.4. Reemplazar la población con nuevos individuos.

Paso 4: Obtener el conjunto de soluciones.

El conjunto de soluciones alcanzado debe cumplir dos características para ser considerado útil: los puntos deben estar lo más cerca posible de la frontera de Pareto exacta (convergencia) y estar distribuidos uniformemente (diversidad). La proximidad a la frontera de Pareto garantiza que los individuos sean soluciones óptimas, mientras que una distribución uniforme de las soluciones significa una buena exploración del espacio de búsqueda y no se dejan regiones sin explorar. Para hacer frente a estos objetivos, las metaheurísticas multiobjetivo incorporan mecanismos para promover la diversidad.

Basándose en la filosofía de Andrew V. Goldberg, los Algoritmos Genéticos (GA) diseñados para resolver Problemas de Optimización Multiobjetivo (MOOPs) deben incluir un mecanismo basado en clasificación no dominada dentro del procedimiento de selección, combinado con una técnica de nicho, lo que proporcionaría una cobertura más amplia del espacio de soluciones al introducir soluciones no dominadas y diversas.

2.2 NSGA-II algorithm

2.2.1 Fast non-dominated Sorting Approach (Intensificación)

El procedimiento de ordenación no dominada original supone un alto coste computacional en cada iteración. Este algoritmo usa una aproximación rápida de ordenación para dividir la población en diferentes niveles de “no dominancia” para poder lidiar con la complejidad. Esta aproximación rápida se lleva a cabo calculando dos entidades para cada solución: primero, el número de soluciones que dominan a la solución que se esté evaluando y, segundo, un conjunto de soluciones dominadas por esa misma solución que se esté evaluando. A cada solución se le asigna un rango igual a su nivel de no dominancia de tal forma que cuanto menor sea, mayor es su nivel de no dominancia y, por tanto, mejor solución es.

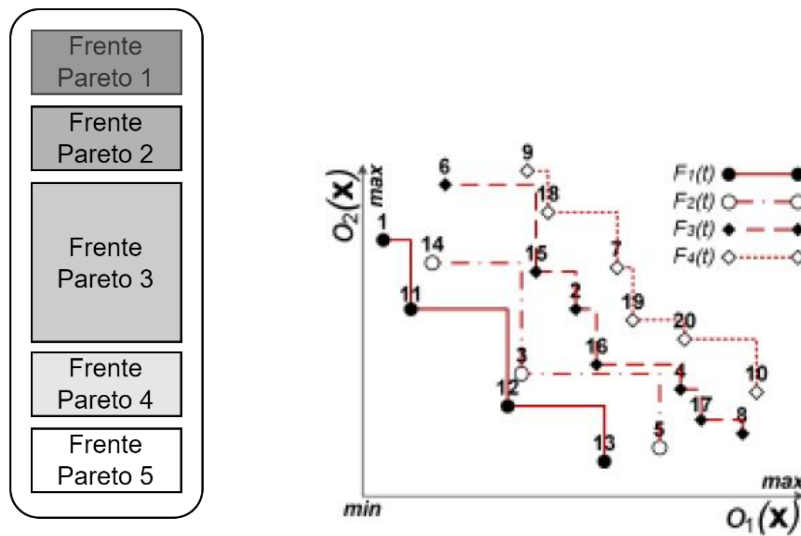


Figura 1: Ordenación por no-dominancia (frentes de Pareto)

2.2.2 Diversity preservation (Diversificación)

Los mecanismos que aseguran diversidad poblacional en la primera versión del algoritmo (NSGA) recaen generalmente en el concepto de *sharing*. Este valor de *sharing* es fijado por el usuario y tiene 2 inconvenientes: 1) El rendimiento del *sharing* depende en gran parte en el valor escogido y 2) la complejidad incrementa rápidamente ya que cada solución tiene que ser comparada con todo el resto de las soluciones de la población.

Con el fin de evitar dicha complejidad y mantener la diversidad, NSGA-II implementa una *crowded comparison* en su lugar. Todos los miembros en la población no dominada tienen asignada una *crowding distance*, que es una medida de la densidad de soluciones en un vecindario.

La *crowding distance* se define como la suma de las distancias entre las soluciones vecinas en cada dimensión del espacio objetivo. Específicamente, se mide cuánto espacio hay entre los vecinos de una solución en el frente de Pareto. Las soluciones con mayor densidad tendrán valores de *crowding distance* más bajos, lo que indica que están rodeadas por otras soluciones cercanas en el frente de Pareto. Por el contrario, si hay menor densidad en el área, las soluciones

tendrán un valor de crowding distance mayor, indicando que son soluciones que ocupan regiones más dispersas en el frente de Pareto. Estas últimas soluciones promueven la diversidad en el conjunto de soluciones de cada generación.

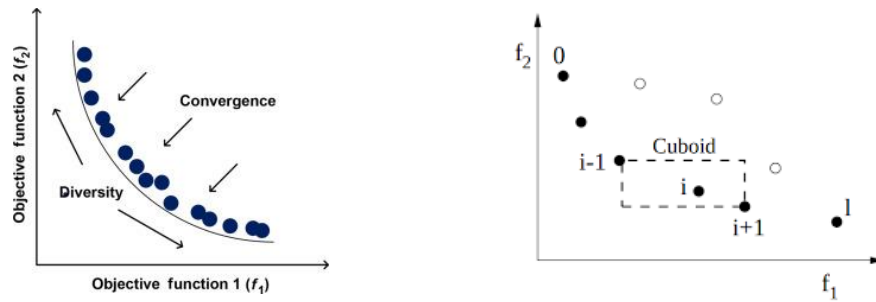


Figura 2: Cálculo de la crowding distance. Diversidad.

2.2.3 Main loop

Las soluciones iniciales se inicialización de manera random. Estas soluciones son evaluadas con la función objetivo diseñada para calcular la bondad de la solución del problema. Con el fin de generar la siguiente generación, se seleccionan los padres utilizando el método de selección por torneo, *tournament selection*. Este método consiste, como se muestra en la figura 3, en seleccionar aleatoriamente dos soluciones y seleccionar entre ambas la solución que tiene un mejor ranking en los frentes de Pareto. En caso de pertenecer al mismo frente de Pareto, se compara la crowding distance y se escoge el individuo con mayor distancia, obtenido un padre para generar la descendencia. Este proceso se repite obteniendo un nuevo padre, que junto con el padre anterior generará la descendencia tras aplicar los operadores de cruce y mutación. Esta selección de padres se realiza hasta obtener tantas parejas de padres como sean necesarias para generar una descendencia del mismo tamaño de la generación actual, de la *population size*.

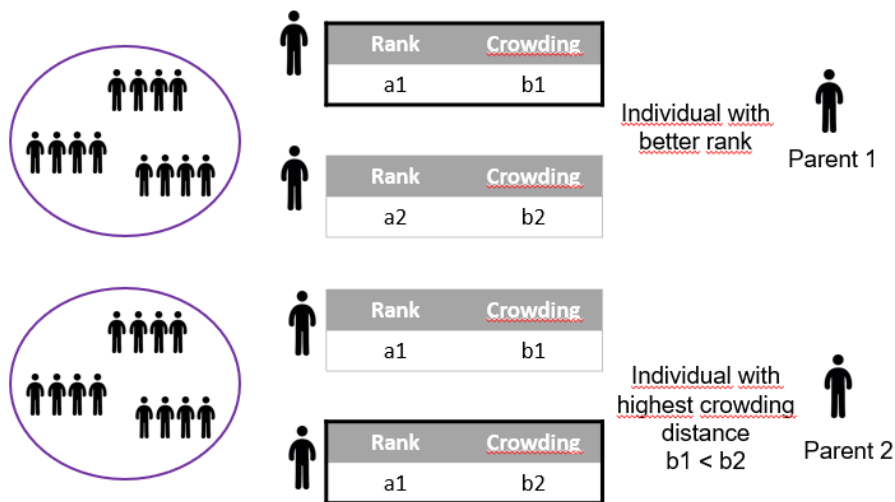


Figura 3: NSGA-II. Selección de padres con selección por torneo.

Tras seleccionar los padres, se procede a generar la descendencia con los operadores de selección y mutación, obteniendo tras combinar con la población actual, una población con el doble de individuos que los permitidos en cada iteración. Por ello, se ha de seleccionar los

individuos que formarán parte de la siguiente generación aplicando tanto la ordenación por no dominancia y la *crowding distance*, asegurando mantener buenas y diversas soluciones en la población.

¿Cómo se realiza esta selección?

Una vez combinados los individuos de la generación con la descendencia, se ordena mediante la ordenación por no dominancia teniendo en cuenta sus valores en las funciones a evaluar y se seleccionan los frentes más cercanos al frente de Pareto en orden de proximidad. Se seleccionan todos los individuos de los frentes hasta encontrar un frente donde la cantidad de individuos del frente sobrepasa la cantidad de individuos que faltan por alcanzar la *population size*. Tal como se puede observar en la figura 4, los individuos del frente de Pareto 1 y del frente de Pareto 2 serán seleccionados para formar parte de la siguiente generación. De esta manera se mantienen las mejores soluciones encontradas en la siguiente generación. La inserción de las soluciones del frente de Pareto 3 en la siguiente generación genera un tamaño de población mayor al permitido, por lo que, en este caso, se aplica la técnica del *crowding distance* para seleccionar una cantidad de soluciones permitidas entre todas las soluciones del frente de Pareto 3. Las soluciones que pasan a la siguiente generación son aquellas con mayor *crowding distance*, es decir, las soluciones que más alejadas del resto se encuentran, promoviendo así la diversidad.

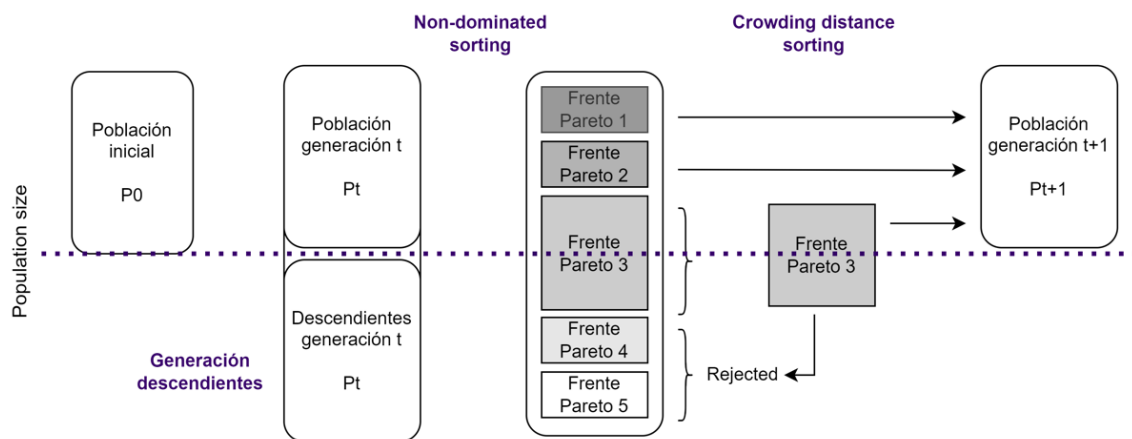


Figura 4: Algoritmo NSGA-II. Proceso completo.

2.3 Implementación

La implementación del algoritmo NSGA-II se ha llevado a cabo utilizando la librería *jmetalpy* en el lenguaje de programación de python. Esta librería incluye tanto la propia implementación del NSGA-II como los tests de Zitzler (ZDT) [2] que se utilizan para comprobar su eficacia. A continuación, se muestra una breve descripción de los test mencionados:

- ZDT1 → Frente de Pareto convexo.
- ZDT2 → Introduce no linealidades y discontinuidad en el frente de Pareto.
- ZDT3 → Presenta concavidades en regiones de Pareto y un frente de Pareto discontinuo.

- ZDT4 → Incorpora no convexidades.
- ZDT5 → Introduce la presencia de variables binarias.
- ZDT6 → Incluye una función no lineal en una de las variables y presenta una región de Pareto más compleja.

Cabe mencionar, que el test de Zitzler número 5, no está disponible en la versión actual de jmetalpy. Con el fin de lidiar con este problema, se han modificado varios archivos de la librería para incorporar el problema y así poder implementar el algoritmo sobre el test número 5. Aunque esto ha posibilitado la implementación del algoritmo, los resultados no se han podido comparar con las soluciones óptimas del Pareto Front, debido a la indisponibilidad.

Los parámetros utilizados para alcanzar las soluciones cercanas a las óptimas tras su optimización son los siguientes:

- Population size = 50
- Offspring size = 50
- Mutación = Polinomial mutation con probabilidad 0.5.
- Cruce = Simulated Binary crossover con probabilidad 1.
- Criterio de parada = un máximo de 15000 evaluaciones.

Para la optimización de los parámetros se ha tenido en cuenta tanto la calidad de las soluciones como el coste computacional que suponen.

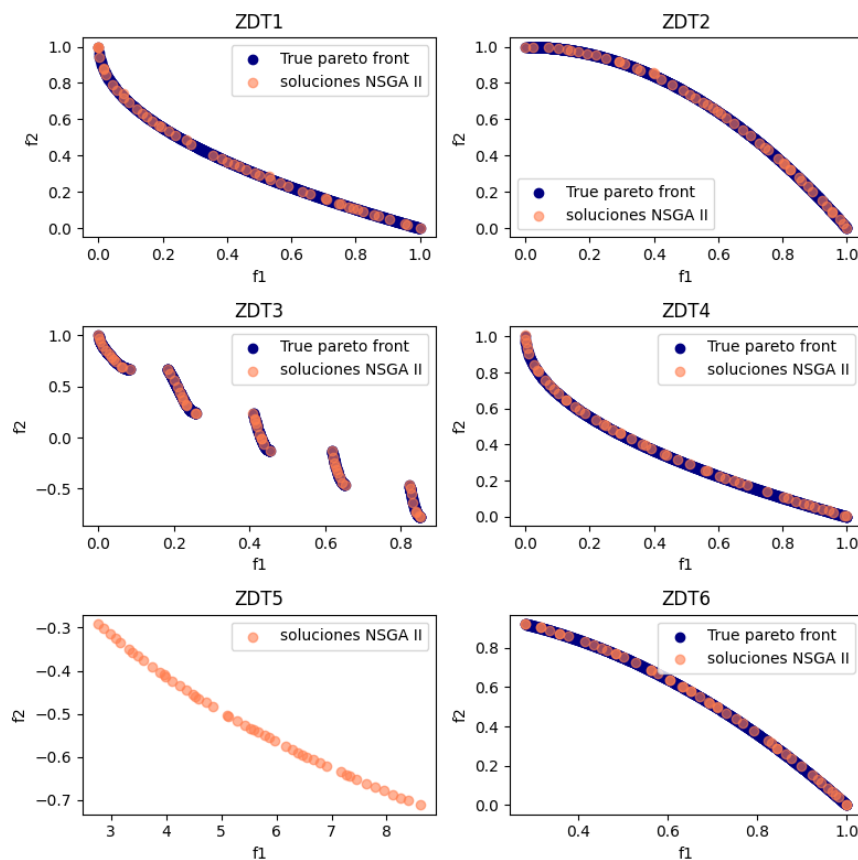


Figura 5: Comparación resultados del algoritmo NSGA-II con el pareto front en los test de Zitzler.

2.4 Conclusiones

El algoritmo NSGA-II es considerado adecuado y efectivo para abordar problemas de optimización multiobjetivo por varias razones. En primer lugar, utiliza un enfoque de ordenación de no dominancia para organizar las soluciones en frentes de Pareto, buscando la intensidad de las soluciones. Por otro lado, la *Crowding Distance* se utiliza para evaluar la diversidad de las soluciones en el frente de Pareto, promoviendo la cobertura en el espacio objetivo y manteniendo soluciones diversas en el frente de Pareto.

Asimismo, se incorpora un mecanismo de elitismo, que preserva las mejores soluciones en cada generación, ayudando a mantener soluciones de alta calidad a lo largo de las iteraciones.

La combinación de elitismo, ordenación por no dominancia y el *crowding distance* permite que NSGA-II realice una buena exploración del espacio de búsqueda al tiempo que explora y explota las soluciones óptimas.

Respecto a la implementación y su uso con el conjunto de problemas de *Zitzler*, destacamos la facilidad de uso de la implementación encontrada en la librería *jmetalpy*, con la única salvedad de que no posee la implementación del quinto problema de forma nativa. Después de realizar un proceso de optimización de los parámetros para la resolución del problema podemos afirmar que se trata de una implementación eficiente e incluso altamente escalable, que podría llegar a usarse en posibles casos de uso reales sin problema alguno con capacidad de enfrentarse con los desafíos analizados en las pruebas.

3 Noising methods

3.1 Introducción

Los *Noising Methods* (NM) [3] son un tipo de metaheurística utilizada en problemas de optimización combinatoria, caracterizados porque involucran la adición de ruido aleatorio a los datos de entrada de un problema, y luego aplican un algoritmo de descenso para encontrar un mínimo local. Este proceso se repite varias veces con diferentes niveles de ruido, y la mejor solución encontrada en todas las iteraciones se devuelve como la solución final. En casos de parametrización específicos, NM puede considerarse como una generalización del algoritmo de recocido simulado (SA).

NM pertenece a una familia de algoritmos de búsqueda local donde se introduce un parámetro de ruido, que permite controlar el proceso de búsqueda para ayudarlo a escapar de óptimos locales. A la hora de implementar un algoritmo específico de NM para un problema concreto, dos de los elementos, o parámetros, más relevantes a definir son la estrategia de introducción de ruido y la estrategia de reducción de ruido.

3.2 Procedimiento

3.2.1 Parámetros de los métodos de ruido

Como mencionamos previamente, los principales parámetros a definir son:

- **El número de intentos.** También es un criterio de parada, lo que significa que el método termina cuando se completa este número de intentos.
- **La tasa de ruido,** que caracteriza la intensidad del ruido
- **El decrecimiento del ruido,** que determina la estrategia de reducción de la intensidad del ruido de forma gradual a medida que avanzan las iteraciones. Esta es una de las similitudes con el recocido simulado, puesto que se aboga por comenzar permitiendo una mayor exploración del espacio de soluciones al inicio para luego ir lentamente convergiendo a una región donde intensificar.

3.2.2 Estrategias de introducción de ruido

Existen múltiples formas de introducir ruido en el procedimiento del algoritmo; a continuación, se presentan las tres visiones generales presentes en el artículo original:

A) Introducción de ruido en la evaluación de la función objetivo

Al valor resultante de la evaluación de una posible solución en la función objetivo se le añade una cierta cantidad arbitraria, dependiente del parámetro establecido, en forma de ruido. De esta forma, se pueden llegar a considerar soluciones “malas” porque pueden poseer menos ruido que una solución “mejor” a la que se le haya añadido una mayor cantidad de ruido. La probabilidad de introducir el ruido se obtiene de una distribución de probabilidad predefinida como, por ejemplo, una gaussiana o una uniforme.

Esto es similar al criterio de aceptación utilizado en el recocido simulado, donde un movimiento malo puede ser aceptado con cierta probabilidad determinada por un parámetro de temperatura. El artículo sugiere que la distribución de probabilidad del ruido debe ser elegida

para contener tanto valores positivos como negativos, de modo que se puedan explorar tanto transformaciones malas como buenas.

B) Modificar los datos originales

Dependiendo de las estructuras de datos usadas y del tipo de dato que contengan, se plantea alterar los datos de entrada de alguna manera. Por ejemplo, en entornos de grafos se podrían añadir valores aleatorios a los pesos de los arcos, o en entornos de vectores numéricos se podrían intercambiar valores de lugar o alterar el propio almacenado en una posición concreta. La estrategia a emplear depende exclusivamente del problema concreto y cómo afecta el ruido, pudiendo plantearnos algunas de las siguientes cuestiones: ¿Es relevante la posición de un elemento en un vector?, ¿Se pueden repetir valores?, ¿Deben tener todos los datos el mismo tamaño?, ...

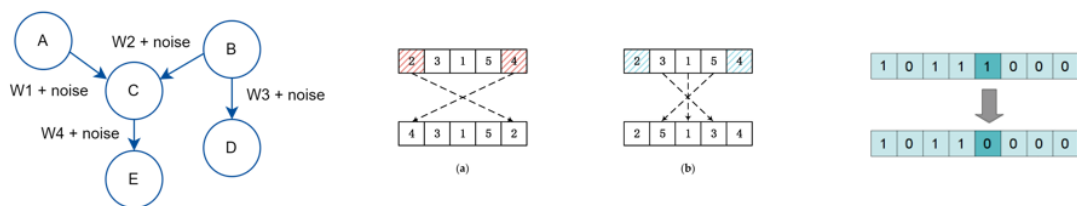


Figura 6: Métodos para añadir ruido a los datos originales.

C) Transformaciones genéricas elementales

La tercera vía consiste en alterar las posibles soluciones que vaya contemplando el algoritmo durante su ejecución. Para mantener un sistema unificado, se plantea el uso de transformaciones genéricas elementales que, como su nombre indica, son unas operaciones predefinidas que se pueden aplicar por igual a cualquier solución que encontremos para obtener otra solución diferente, generando así un mismo nivel de ruido cada vez que se apliquen, siempre y cuando se trate de la misma transformación. La única diferencia entre las transformaciones genéricas que se muestran a continuación son los parámetros que las definen y aquí se presentan las principales transformaciones genéricas en el ámbito de las metaheurísticas:

- **Sustitución** (o complementación para las soluciones codificadas como cadenas binarias). Consiste en reemplazar un carácter en la solución por otro carácter del mismo alfabeto, asegurando que la solución resultante siga siendo válida.
- **Intercambio** La transformación de intercambio implica intercambiar las posiciones de dos caracteres en la solución actual. Específicamente, intercambia los dos caracteres ubicados en las posiciones a y b en la solución actual, donde a y b son parámetros.
- **Desplazamiento**. La transformación de desplazamiento implica insertar un carácter de la solución actual en una nueva posición y luego desplazar los caracteres ubicados entre las posiciones antigua y nueva. Específicamente, la transformación de desplazamiento con parámetros a y b inserta el carácter ubicado en la posición b en la posición a, y luego desplaza los caracteres ubicados entre las posiciones a (incluida) y b (excluida).

- **Inversión.** La transformación de inversión implica invertir el orden de los elementos ubicados entre dos posiciones en la solución actual. Específicamente, la transformación de inversión con parámetros a y b invierte el orden de los elementos ubicados entre las posiciones a y b en la solución actual.

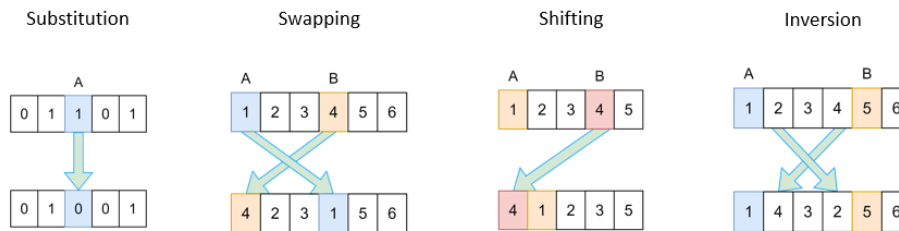


Figura 7: Transformaciones elementales. Substitution, swapping, shifting, inversion.

3.2.3 Estrategias de decrecimiento del ruido

A) Aritmético

Esta estrategia de decrecimiento viene dada por la siguiente fórmula: $\alpha = \frac{r_{min} - r_{min}}{G - 1}$,

Donde α es el factor de decrecimiento, r_{min} y r_{max} son los valores mínimo y máximo de ruido, y G es el número total de generaciones.

B) Geométrico

Esta otra alternativa consiste en directamente multiplicar el valor de ratio de ruido (r) por un factor de reducción de ruido parametrizado de antemano (α), siguiendo la siguiente fórmula:

$$r * \alpha$$

3.2.4 Exploración de los entornos

El conjunto de soluciones obtenido al aplicar una misma transformación a una solución inicial, con diferentes valores de parámetros, se denomina entorno. Cada transformación dará un entorno diferente para una misma solución inicial, de tal forma que definiendo el entorno de una solución se puede diseñar un proceso iterativo de optimización local. Este proceso puede llegar a ser muy costoso computacionalmente dependiendo de la estrategia usada y por ello existen varias aproximaciones que buscan un equilibrio entre el coste asociado a la exploración y los beneficios de una exploración más minuciosa:

- **Exploración Aleatoria.** Esta estrategia implica explorar el espacio de soluciones seleccionando aleatoriamente soluciones del vecindario de la solución actual. La principal ventaja de esta estrategia es que puede muestrear el espacio de soluciones más eficientemente que otras estrategias, especialmente cuando el tamaño del vecindario es muy grande. Sin embargo, el inconveniente es que puede no ser capaz de aprovechar los intentos previos para reducir la complejidad amortizada, y puede intentar la misma solución varias veces en lugar de explorar otras partes del vecindario.

- **Exploración Sistemática.** Esta estrategia implica explorar el espacio de soluciones probando los diferentes valores de los parámetros de las transformaciones elementales de manera sistemática. La principal ventaja de esta estrategia es que puede encontrar el primer mejor vecino más eficientemente que otras estrategias, pero puede ser demasiado determinista y no ser capaz de explorar el espacio de soluciones exhaustivamente.
- **Exploración Exhaustiva.** Esta estrategia implica explorar el espacio de soluciones probando todas las posibles soluciones en el vecindario de la solución actual. La principal ventaja de esta estrategia es que garantiza encontrar la mejor solución en el vecindario, pero puede tardar demasiado, especialmente cuando el tamaño del vecindario es muy grande.

3.3 Variantes

Aquí se presentan dos variantes propuestas en el artículo original que plantean variaciones en aspectos diferentes.

- La primera propone alternar fases de ruido con fases sin ruido. Se ejecutan iteraciones sin ruido hasta alcanzar un óptimo local, momento en el cual se establecen un valor de ruido para las siguientes NT (*noised trials*) iteraciones. Además, el valor del ruido decrecerá a lo largo del tiempo, pero mientras se esté aplicando será constante.

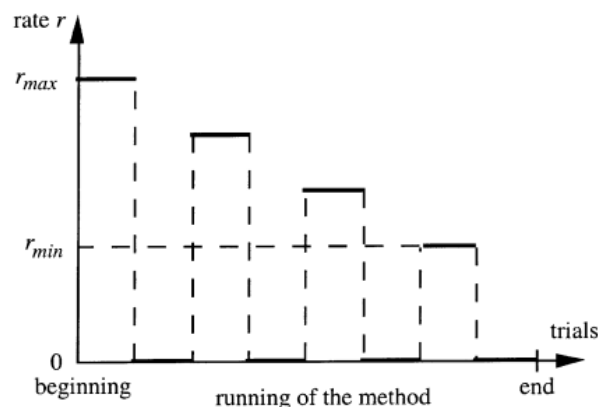


Figura 8: Primera variante de noising methods Alternar fases de ruido y fases sin ruido.

- La segunda variante consiste en volver a la mejor solución encontrada periódicamente. Teniendo en cuenta que hay soluciones peores que pueden ser aceptadas, es posible un paso hacia una solución peor, dejando en el camino una solución interesante. Por lo que se propone reinicializar la solución actual con la mejor solución encontrada hasta el momento de forma periódica. La frecuencia de inicialización se establece a un parámetro α , indicando las iteraciones en las que reiniciar las soluciones.

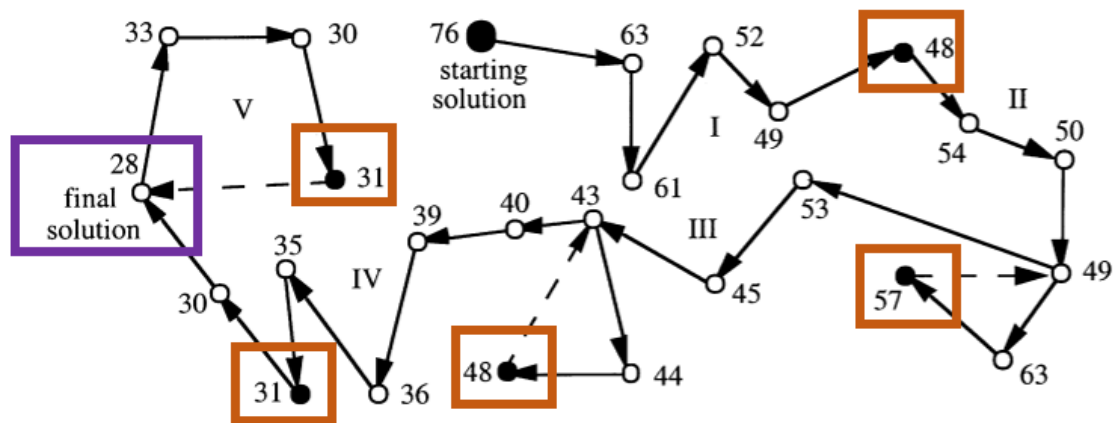


Figura 9: Segunda variante noising methods. Periódicamente volver a la mejor solución encontrada.

3.4 Aplicaciones

Los *Noising Methods* [3] han sido aplicados a una gran variedad de problemas de optimización combinatoria incluyendo el problema del viajante [4], particionado de grafos [6], el problema de la mochila [7], y el problema de la asignación de tareas [8]. Han demostrado ser efectivos en la búsqueda de soluciones próximas al frente de Pareto, especialmente al ser combinadas con otras metaheurísticas como el Recocido Simulado [11] o la Búsqueda Tabú [10].

3.5 Conclusiones

En resumen, los Noising Methods (NM) son una metaheurística eficaz para problemas de optimización combinatoria. Introducen ruido aleatorio en los datos de entrada, aplican un algoritmo de descenso y repiten el proceso con diferentes niveles de ruido. Aspectos clave incluyen estrategias de introducción y reducción de ruido, así como exploración de entornos. Las variantes, como alternar fases de ruido y reinicialización periódica, mejoran la flexibilidad del método. NM ha demostrado éxito en problemas como el viajante, particionado de grafos, la mochila y asignación de tareas, siendo efectivos junto con otras metaheurísticas. En conclusión, los Noising Methods son una herramienta prometedora y versátil para abordar desafíos de optimización combinatoria.

4 Hormigas Max-Min

4.1 Introducción

La investigación sobre la Optimización de Colonias de Hormigas (ACO) emplea una estrategia voraz a la hora de intensificar las soluciones, lo que puede llevar al estancamiento en soluciones de baja calidad. Para evitar esto, el algoritmo Min-Max Ant System (MMAS) [9] propone juntar una explotación mejorada de las mejores soluciones encontradas durante la búsqueda, con un mecanismo efectivo para evitar el estancamiento temprano de la búsqueda. Este sistema MAX-MIN, difiere en tres aspectos clave respecto al sistema de hormigas convencional:

- i. Después de cada iteración, solo una hormiga agrega feromonas. Así, explotamos las mejores soluciones encontradas. El criterio de selección de esa hormiga puede ser uno de los siguientes:
 - a. La que encontró la mejor solución en la iteración actual (mejor hormiga de la iteración)
 - b. La que encontró la mejor solución desde el inicio de la ejecución (mejor hormiga global)
- ii. Para evitar que la búsqueda se estanque, el rango de feromonas en cada componente de la solución se limita a un intervalo $[\tau_{min}, \tau_{max}]$.
- iii. Además, se inicializan deliberadamente las sendas de feromonas a τ_{max} , logrando de esta manera una mayor exploración de soluciones al inicio del algoritmo (de forma similar al recocido simulado).

4.2 Algoritmo Hormigas Max-Min

4.2.1 Pheromone trail updating

En MMAS se utiliza una sola hormiga para actualizar las feromonas después de cada iteración. En consecuencia, la regla modificada de actualización de los rastros de feromona viene dada por

$$\tau_{ij}(t + 1) = p\tau_{ij}(t) + \Delta\tau_{ij}^{best}$$

donde $\Delta\tau_{ij}^{best} = 1/f(S^{best})$ y $f(S^{best})$ denota el coste de la mejor solución de la iteración o de la mejor solución global. Gracias a esto, los elementos de la solución que aparecen con frecuencia en las mejores soluciones obtienen un gran refuerzo.

Si solo se utiliza la mejor solución global, la búsqueda puede converger demasiado rápido alrededor de la solución y quedar limitada la exploración de otras soluciones, con la consecuencia de quedar atrapados en óptimos locales.

Este peligro se reduce cuando se elige la mejor solución de la iteración para la actualización del rastro de feromonas, ya que las mejores soluciones de la iteración pueden diferir considerablemente de iteración a iteración y un mayor número de componentes pueden recibir refuerzos ocasionales.

O lo que es lo mismo, la selección de la mejor solución a nivel global intensifica en ese espacio de búsqueda mientras que la elección de la mejor solución en cada iteración diversifica y promueve la exploración en diferentes espacios de búsqueda.

Por tanto, parece lógico utilizar estrategias mixtas, como elegir las mejores soluciones de las iteraciones por defecto para actualizar las feromonas y utilizar la mejor solución global sólo cada cierto número fijo de iteraciones.

4.2.2 Pheromone trail limits

Independientemente de la elección de la mejor hormiga (global o de iteración) para la actualización del rastro de feromonas, puede producirse un estancamiento de la búsqueda. Esto puede ocurrir si en cada punto de elección, el rastro de feromonas es significativamente mayor para una elección que para todas las demás. En esta situación, teniendo en cuenta la probabilidad de elección,

$$p_{ij}^k(t) = \frac{[\tau_{ij}(t)]^\alpha [\eta_{ij}]^\beta}{\sum_{l \in N_i^k} [\tau_{il}(t)]^\alpha [\eta_{il}]^\beta} \text{ if } j \in N_i^k$$

una hormiga preferirá esta solución sobre todas las alternativas y se le dará refuerzo adicional a la solución en la actualización de las feromonas. Algo similar, pero a la inversa, ocurriría si el valor de las rutas no seleccionadas comienza a decrecer rápidamente: las rutas que podríamos emplear para diversificar comienzan a ser inaccesibles por su extremadamente bajo coste.

En tal situación, las hormigas podrían llegar a construir la misma solución una y otra vez y la exploración del espacio de búsqueda se detendría.

¿Cómo evitarlo?

Limitando la influencia de las feromonas se puede evitar que las diferencias relativas de las feromonas entre las diferentes soluciones sean muy extremas. Para ello, MMAS impone límites τ_{min} y τ_{max} en la cantidad mínima y máxima de feromonas. Por lo que, en cada iteración, se ha de asegurar que la cantidad de feromonas se encuentra dentro del rango determinado, estableciendo los valores τ_{min} en los casos donde no se alcanza el mínimo y τ_{max} en caso de superar el máximo.

¿Cómo determinar el valor τ_{max} ?

Convergencia $\rightarrow$ si para cada elección, la “opción correcta” tiene como rastro de feromona τ_{max} asociado, mientras que el resto de las soluciones tiene un rastro de feromona τ_{min} .

La convergencia de MMAS difiere ligeramente en el concepto de estancamiento. El estancamiento se refiere a la situación en la que todas las hormigas siguen el mismo camino, en cambio, en las situaciones de convergencia de MMAS no ocurre lo mismo debido al uso de los límites del camino de feromonas. El valor máximo de feromonas a añadir en cada iteración es $1/f(S^{opt})$ donde $f(S^{opt})$ es el valor de la solución óptima. En MMAS, se utiliza $f(S^{gb})$ (el valor en la función objetivo de la mejor solución encontrada hasta el momento de forma global) en lugar de $f(S^{opt})$ para fijar el rastro máximo de feromona τ_{max} a una estimación del valor

máximo asintótico, ya que el valor de la solución óptima es desconocido. Por lo tanto, y teniendo en cuenta la ecuación del nivel de feromona descontado, como máximo el nivel de feromonas τ_{max} es $1/(1-p) * 1/(f(S^{gb}))$, siendo p un factor constante de feromonas que permanecen en el proceso de evaporación de feromonas. Este valor, τ_{max} , se actualiza cada vez que se encuentra una mejor solución, lo que conduce a un valor de $\tau_{max}(t)$ que cambia dinámicamente.

¿Cómo determinar el valor τ_{min} ?

Primero, partamos de dos asunciones:

- 1- Las soluciones que mejoran a la actual se encuentran, de forma generalizada, cerca de la solución actual.
- 2- Lo que más influye en la construcción de soluciones es la diferencia relativa entre τ_{min} y τ_{max} , en vez de las diferencias relativas de información heurística

Teniendo esto en cuenta, se pueden calcular buenos valores de τ_{min} a partir de la relacionarlos con la convergencia del algoritmo. Para ello, establecemos que la solución óptima se construirá con una probabilidad (parametrizable) p_{best} que, a su vez se entiende como la probabilidad de elección de la mejor opción en cada paso de decisión p_{dec} de forma consecutiva, de tal forma que $p_{best} = p_{dec}^n$.

Si asumimos que escogemos el camino con mayor valor de feromona podemos generalizar la ecuación (índice a la primera ecuación que tenemos) de la siguiente forma:

$$p_{dec} = \frac{\tau_{max}}{\tau_{max} + (avg - 1)\tau_{min}}$$

Donde avg es el valor medio de posibles elecciones que realiza una hormiga en cada punto de decisión ($avg = n/2$). Esto es así porque asumimos la convergencia del algoritmo, es decir, que el camino escogido tiene el valor de feromonas igual a τ_{max} y que existe un conjunto de alternativas no escogidas, de tamaño $avg - 1$, cuyo valor de feromonas asociado es τ_{min} .

Si sustituimos p_{dec} por $\sqrt[n]{p_{best}}$ y despejamos para τ_{min} , obtenemos la siguiente fórmula:

$$\tau_{min} = \frac{\tau_{max}(1 - p_{dec})}{(avg - 1)\sqrt[n]{p_{best}}}$$

De tal forma que podemos calcular en cada iteración el valor mínimo de feromonas a partir de límite máximo y el parámetro p_{best} . Si $p_{best} = 1$, entonces $\tau_{min} = 0$ y si por el contrario es demasiado pequeño, puede suceder que $\tau_{min} > \tau_{max}$, en cuyo caso se fijará $\tau_{min} = \tau_{max}$. Por tanto, se emplean valores de p_{best} cercanos a 1 pero no iguales con el objetivo de

4.2.3 Inicialización de rastro de feromonas

Al inicio de la ejecución, establecemos los valores de feromonas de cada ruta a un valor superior al límite máximo permitido τ_{max} , de tal forma que, después de la primera iteración, el valor de feromona de cada ruta sea igual al máximo. Adicionalmente, durante las primeras iteraciones

del algoritmo se mantendrá este valor de feromona con el objetivo de lograr una exploración muy amplia durante las primeras iteraciones.

4.3 Implementación

El problema en el que vamos a realizar la implementación consiste en asignar un conjunto de tareas a un conjunto de operarios, de tal manera que se minimice el costo total de la asignación. Cada tarea tiene un tiempo de ejecución y cada operario tiene un tiempo disponible para realizar actividades. Además, existe un costo asociado a la asignación de cada tarea a cada operario. El objetivo es encontrar la asignación óptima que minimice el costo total y garantice que todas las tareas se completen. Se va a implementar la metaheurística Sistema de Hormigas Max-Min para encontrar soluciones aproximadas en un tiempo razonable. Tanto los costes que tienen asignados cada uno de los operarios para cada una de las tareas, como el tiempo disponible de cada uno de los operarios, como el tiempo requerido de cada tarea vienen dados por la tabla que podemos observar en la figura 10.

	T_1	T_2	T_3	T_4	Tiempo disponible
O_1	2	3	4	1	4
O_2	3	2	3	2	5
O_3	2	2	1	2	3
O_4	3	3	3	3	4
O_5	2	1	2	2	4
Tiempo requerido	3	2	2	3	

Figura 10: Valores del problema de asignación

Para implementar este problema con un sistema de colonia de hormigas se adapta el problema a un grafo ya que estas metaheurísticas están pensadas para la optimización de grafos.

Para adaptar este problema, se plantea el uso de un grafo bipartito, es decir, un grafo con nodos que pertenecen a una de dos clases diferentes. Cada arco de un grafo bipartito siempre une dos nodos de clases diferentes, de forma que no haya nodos que estén unidos por un arco a otros nodos de su clase. En este caso concreto, se define que unos nodos serán de la clase operario y los otros de la clase tarea. También, se plantea un grafo dirigido, es decir, que cada uno de los arcos tuviera una dirección concreta.

En nuestro caso, todos los nodos del tipo operario tendrán un arco que vaya en dirección a cada uno de los nodos tarea y viceversa. Los arcos que van de las tareas a los operarios tendrán un valor dado por la matriz de coste mientras que los que van de los operarios a las tareas tendrán peso 0. De esta forma, una tarea se considera “asignada” a un operario cuando la hormiga transite por el arco que va desde el nodo de la tarea al nodo del operario. En caso de plantear la asignación de forma inversa, es decir, interpretando la asignación como la transición de un nodo operario a un nodo tarea, aumenta innecesariamente la complejidad tanto de la implementación como del coste computacional. Es decir, teniendo en cuenta que un operario puede tener muchas tareas y que una tarea únicamente un operario, se escogen las tareas de manera aleatorio, y a partir de las tareas se selecciona el operario que realiza la tarea. En caso de comenzar por los operarios, habría que asignarles tareas a los operarios siempre comprobando que no se incumplan las restricciones.

Para poder implementar este problema hay que realizar una pequeña modificación a los sistemas de colonias de hormigas, ya que, en principio, estos no permiten a una hormiga pasar dos veces por el mismo nodo. Para este problema, se define que un operario puede realizar múltiples tareas, mientras tenga tiempo disponible, pero una tarea solo puede ser realizada por un único operario. Por esto, tenemos que mantener la restricción original para los nodos de tipo tarea, pero eliminarla para los nodos de tipo operario.

Para el paso de las tareas a los operarios se deben poner otro tipo de restricción ya que, a pesar de poder hacer más de una tarea, los operarios tienen un tiempo de disponibilidad y las tareas un tiempo necesario para completarse, de modo que, si un operario no tiene suficientes horas disponibles para poder completar la tarea del nodo en el que este la hormiga, esta no podrá moverse a dicho nodo de operario.

A pesar de tener ya la estructura del grafo necesaria para la implementación de la metaheurística, se realiza un cambio menor para facilitar el desarrollo del código. Este cambio es la no representación de los arcos de peso 0, es decir, los arcos que van de los operarios a las tareas, en las estructuras de datos empleadas en el algoritmo. De esta forma, una vez una hormiga llegue a un nodo de operario, se coloca directamente en un nodo tarea que no haya visitado, elegido al azar. Esto se hace, para empezar, con el objetivo de evitar divisiones entre 0 a la hora de calcular la información heurística y, por otra parte, porque reduce el uso de memoria del algoritmo, provocando que las dimensiones de la matriz de pesos y la matriz de feromonas se reduzcan.

Cabe mencionar que la implementación del algoritmo para el problema propuesto encuentra la solución óptima global, con un valor de 6, en muy pocas iteraciones, dependiendo de la cantidad de hormigas establecidas.

La figura 11 ilustra la evolución durante las iteraciones de la cantidad de feromonas en cada arco, donde cada fila corresponde a las distintas tareas y cada columna los diferentes operarios. En el eje x se puede analizar la cantidad de iteraciones. Se percibe una pronta convergencia de las soluciones, esto se debe a que el tamaño del problema es muy pequeño, y dado a que en cada iteración solo usamos la mejor hormiga de dicha iteración (o la mejor hormiga global en iteraciones tardías), el algoritmo tiende a converger rápido a la primera solución que encuentra dentro de las soluciones óptimas globales.

En la figura 12, asimismo, se puede observar la cantidad de hormigas que pasan por los arcos, lo que es proporcional a la cantidad de feromonas que se depositan en los caminos. Existe un componente aleatorio a la hora de elegir los caminos que siguen las hormigas y se impide que la posibilidad de que una hormiga tome un camino sea nula (salvo que el camino no sea factible). Esto se debe al valor mínimo de feromonas que se determina en el algoritmo.

En ambas figuras se puede observar cómo en las primeras iteraciones los arcos que van de la tarea 1 al operario 5 y de la tarea 2 con el operario 2 tienen una gran cantidad de feromona. No obstante, a partir de la iteración 50 aproximadamente, los arcos que pasan a tener una cantidad mayor de feromona son el que une la tarea 1 con el operario 2 y el que une la tarea 2 con el operario 5. Esto se debe a que las mejores hormigas de la iteración han encontrado un camino más eficiente tomando caminos de menor probabilidad, aumentando, en consecuencia, el nivel

de feromonas de dicho camino, y por lo tanto, aumentando la probabilidad de coger ese camino para el resto de hormigas en las iteraciones posteriores. Si la probabilidad de tomar un camino fuese 0, las hormigas no podrían tomar aquellos caminos de menor probabilidad. No obstante, en este algoritmo no se permite que la feromona de los arcos sea nula por el nivel mínimo de feromona establecido, por lo que posibilita tomar caminos peores.

En problemas más grandes esto puede evitar óptimos locales que, en caso de permitir que la feromona de los arcos fuese nula, no podríamos evitar, ya que imposibilitaríamos a las hormigas tomar caminos diferentes al mejor encontrado hasta el momento. El tener un valor máximo para la feromona también ayuda a la diversificación, ya que, en caso de no asignar un valor máximo, la feromona de un arco podría crecer ilimitadamente haciendo que las probabilidades del resto de arcos disponibles, a pesar de no ser nulas, sean difíciles de escoger.

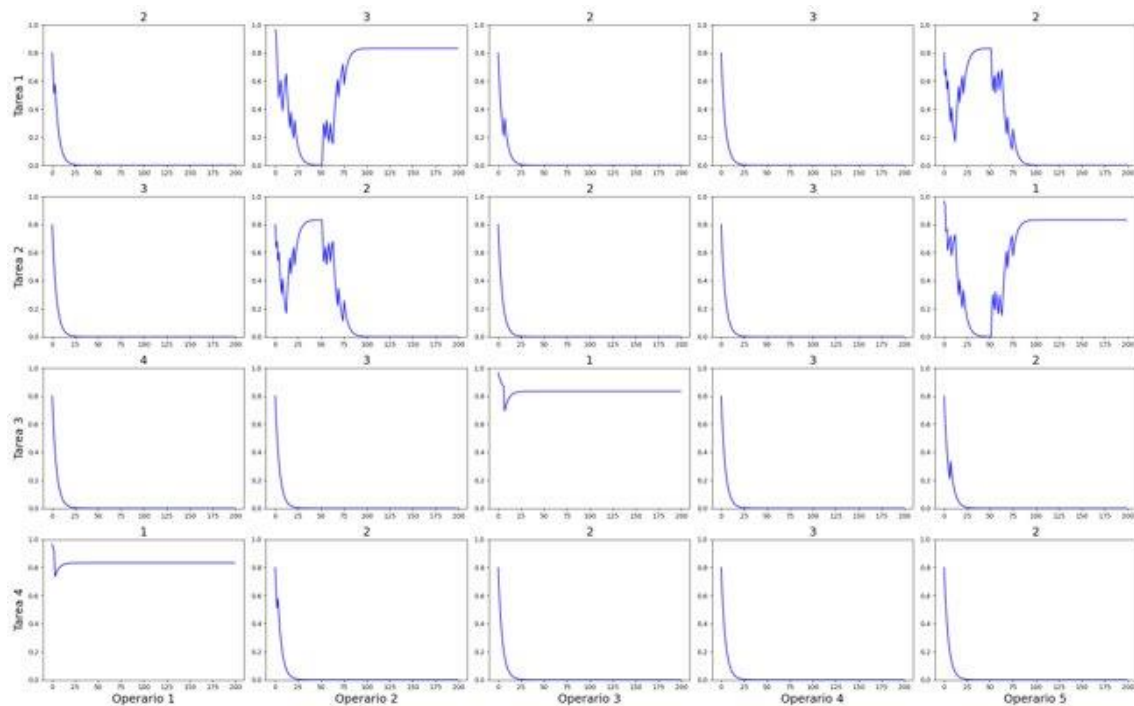


Figura 11: Evolución durante las iteraciones de la cantidad de feromona de cada arco

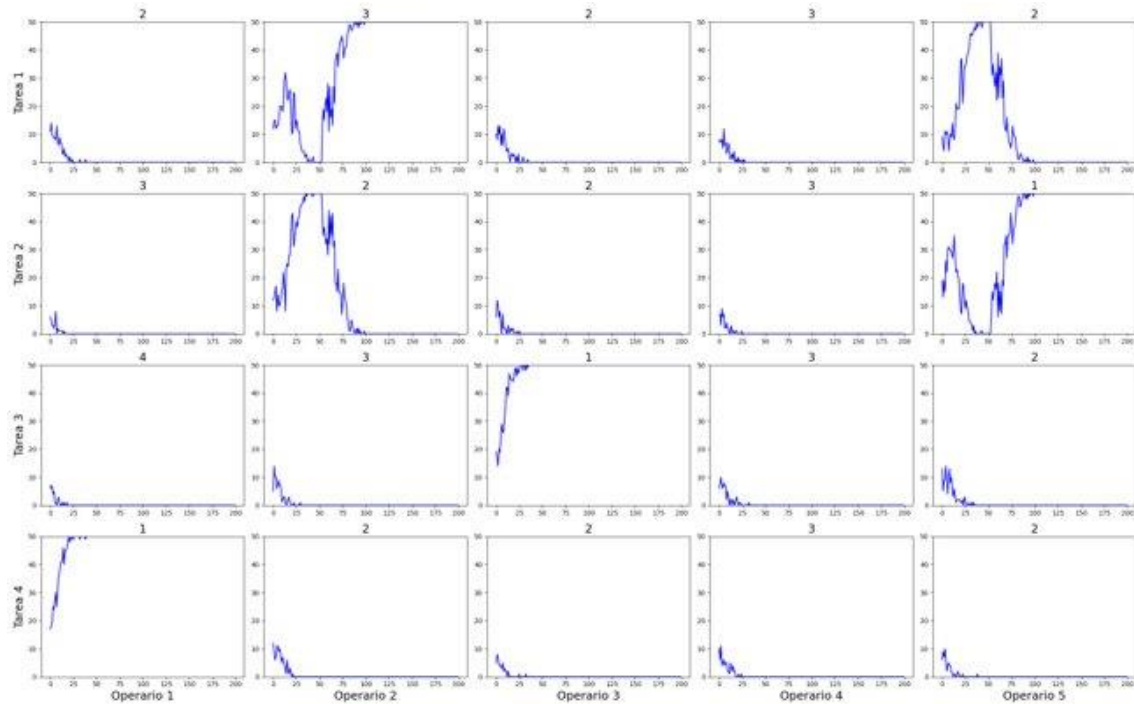


Figura 12: Evolución de la cantidad de hormigas que pasan por cada arco durante las iteraciones

4.4 Conclusiones

En conclusión, la implementación del algoritmo de Hormigas Max-Min para abordar el problema de asignación de tareas a operarios ha resultado en una adaptación eficiente de esta metaheurística. El enfoque Max-Min incorpora elementos clave, como la actualización de los rastros de feromona por una única hormiga y la limitación de los valores de feromona, para mejorar la exploración del espacio de búsqueda y evitar el estancamiento prematuro.

El uso de una estrategia mixta en la selección de la mejor hormiga para la actualización de feromonas, considerando tanto la mejor solución global como la mejor solución de la iteración actual, contribuye a equilibrar la intensificación y la diversificación en la búsqueda. Esto permite que el algoritmo explore diferentes soluciones y evite quedar atrapado en óptimos locales.

La introducción de límites en los valores de feromona, mediante $\tau_{\min}$ y $\tau_{\max}$, juega un papel crucial en la prevención de situaciones de estancamiento y en la promoción de la exploración del espacio de búsqueda. Mientras que la actualización dinámica de $\tau_{\max}$, basada en la mejor solución global encontrada hasta el momento, asegura una adaptación continua a medida que el algoritmo avanza, la inicialización de los rastros de feromona con valores superiores a $\tau_{\max}$ al inicio de la ejecución favorece una exploración amplia en las primeras iteraciones.

En cuanto a la implementación del problema de asignación de tareas a operarios, la representación del problema como un grafo bipartito dirigido ha facilitado la aplicación del algoritmo de Hormigas Max-Min. Se han tenido que realizar ajustes para adaptarlo correctamente a una implementación eficiente, pero manteniendo la estructura del algoritmo original.

5 Conclusiones

El análisis en profundidad de las distintas metaheurísticas analizadas nos han permitido conocer más acerca de las metaheurísticas tanto evolutivas como constructivas. También nos ha permitido estudiar y entender en gran detalle sus implementaciones y la potencia que presentan.

Como ideas comunes a las metaheurísticas nos llevamos la importancia de equilibrio de las fases de diversificación, que nos permiten evaluar diferentes zonas del espacio de soluciones, y de intensificación, como aproximación de la convergencia del algoritmo hacía soluciones de calidad.

Además, la resolución de problemas multiobjetivo resulta de gran utilidad en entornos reales y haber podido profundizar en la adaptación de estas metaheurísticas a esos entornos de mayor complejidad, nos deja con una sensación de haber aprendido conocimientos realmente relevantes y altamente aplicables en la resolución de problemas reales.

Estamos seguros de que todas estas ideas nos serán útiles incluso en otras áreas diferentes a la resolución de problemas de optimización.

6 Bibliografía

- [1] Deb, K., Pratap, A., Agarwal, S., & Meyarivan, T. A. M. T. (2002). A fast and elitist multiobjective genetic algorithm: NSGA-II. *IEEE transactions on evolutionary computation*, 6(2), 182-197.
- [2] Zitzler, E., Deb, K., & Thiele, L. (2000). Comparison of multiobjective evolutionary algorithms: Empirical results. *Evolutionary computation*, 8(2), 173-195.
- [3] Charon, I., & Hudry, O. (2001). The noising methods: A generalization of some metaheuristics. *European Journal of Operational Research*, 135(1), 86-101.
- [4] Charon, I., & Hudry, O. (2000). Application of the noising method to the travelling salesman problem. *European Journal of Operational Research*, 125(2), 266-277.
- [5] Charon, I., & Hudry, O. (1993). The noising method: a new method for combinatorial optimization. *Operations Research Letters*, 14(3), 133-137.
- [6] Charon, I., & Hudry, O. (2006). Noising methods for a clique partitioning problem. *Discrete Applied Mathematics*, 154(5), 754-769.
- [7] Zhan, S., Wang, L., Zhang, Z., & Zhong, Y. (2020). Noising methods with hybrid greedy repair operator for 0–1 knapsack problem. *Memetic Computing*, 12, 37-50.
- [8] Chen, W. H., & Lin, C. S. (2000). A hybrid heuristic to solve a task allocation problem. *Computers & Operations Research*, 27(3), 287-303.
- [9] Stützle, T., & Hoos, H. H. (2000). MAX–MIN ant system. *Future generation computer systems*, 16(8), 889-914.
- [10] Glover, F., & Melian, B. (2003). Tabu search. *Revista iberoamericana de inteligencia artificial*, 19(1), 29-48.
- [11] Van Laarhoven, P. J., Aarts, E. H., van Laarhoven, P. J., & Aarts, E. H. (1987). *Simulated annealing* (pp. 7-15). Springer Netherlands.